

4.7.7 Linear and Poisson Regression on the Bikeshare Data

Ở đây ta fit các linear regression model và Poisson regression model cho dữ liệu **Bikeshare**, như đã mô tả trong Mục 4.6. Response **bikers** đo số lượt thuê xe đạp mỗi giờ tại Washington, DC trong giai đoạn 2010–2012.

In [64]: `Bike = load_data('Bikeshare')`

Ta hãy xem nhanh kích thước và tên các variable trong dataframe này.

In [65]: `Bike.shape, Bike.columns`

Out[65]: `((8645, 15),
Index(['season', 'mnth', 'day', 'hr', 'holiday', 'weekday',
 'workingday', 'weathersit', 'temp', 'atemp', 'hum',
 'windspeed', 'casual', 'registered', 'bikers'],
 dtype='object'))`

Linear Regression

Ta bắt đầu bằng cách fit một linear regression model cho dữ liệu.

In [66]: `X = MS(['mnth',
 'hr',
 'workingday',
 'temp',
 'weathersit']).fit_transform(Bike)
Y = Bike['bikers']
M_lm = sm.OLS(Y, X).fit()
summarize(M_lm)`

Out[66]:

	coef	std err	t	P> t
intercept	-68.6317	5.307	-12.932	0.000
mnth[Feb]	6.8452	4.287	1.597	0.110
mnth[March]	16.5514	4.301	3.848	0.000
mnth[April]	41.4249	4.972	8.331	0.000
mnth[May]	72.5571	5.641	12.862	0.000
mnth[June]	67.8187	6.544	10.364	0.000
mnth[July]	45.3245	7.081	6.401	0.000
mnth[Aug]	53.2430	6.640	8.019	0.000
mnth[Sept]	66.6783	5.925	11.254	0.000
mnth[Oct]	75.8343	4.950	15.319	0.000
mnth[Nov]	60.3100	4.610	13.083	0.000
mnth[Dec]	46.4577	4.271	10.878	0.000
hr[1]	-14.5793	5.699	-2.558	0.011
hr[2]	-21.5791	5.733	-3.764	0.000
hr[3]	-31.1408	5.778	-5.389	0.000
.....				

Có 24 level của **hr** và tổng cộng 40 row, vì vậy ta đã truncate summary. Trong **M_lm**, các level đầu tiên **hr[0]** và **mnth[Jan]** được xem là baseline value, nên không có coefficient estimate nào được báo cáo cho chúng: ngầm hiểu rằng coefficient estimate của chúng bằng 0, và mọi level khác được đo tương đối so với các baseline này. Ví dụ, coefficient 6.845 của **Feb** cho biết rằng, khi giữ mọi variable khác cố định, trung bình có khoảng 7 rider nhiều hơn vào tháng Hai so với tháng Một. Tương tự, có khoảng 16.5 rider nhiều hơn vào tháng Ba so với tháng Một.

Các kết quả thấy trong Mục 4.6.1 dùng một coding hơi khác cho các

variable `hr` và `mnth`, như sau:

```
In [67]: hr_encode = contrast('hr', 'sum')
mnth_encode = contrast('mnth', 'sum')
```

Ta refit lại:

```
In [68]: X2 = MS([mnth_encode,
                 hr_encode,
                 'workingday',
                 'temp',
                 'weathersit']).fit_transform(Bike)
M2_lm = sm.OLS(Y, X2).fit()
S2 = summarize(M2_lm)
S2
```

```
Out[68]:
```

	coef	std err	t	P> t
intercept	73.5974	5.132	14.340	0.000
mnth[Jan]	-46.0871	4.085	-11.281	0.000
mnth[Feb]	-39.2419	3.539	-11.088	0.000
mnth[March]	-29.5357	3.155	-9.361	0.000
mnth[April]	-4.6622	2.741	-1.701	0.089
mnth[May]	26.4700	2.851	9.285	0.000
mnth[June]	21.7317	3.465	6.272	0.000
mnth[July]	-0.7626	3.908	-0.195	0.845
mnth[Aug]	7.1560	3.535	2.024	0.043
mnth[Sept]	20.5912	3.046	6.761	0.000
mnth[Oct]	29.7472	2.700	11.019	0.000
mnth[Nov]	14.2229	2.860	4.972	0.000
hr[0]	-96.1420	3.955	-24.307	0.000
hr[1]	-110.7213	3.966	-27.916	0.000
hr[2]	-117.7212	4.016	-29.310	0.000
.....				

Hai cách coding khác nhau như thế nào? Trong `M2_lm`, coefficient estimate được báo cáo cho mọi level ngoại trừ level 23 của `hr` và level Dec của `mnth`. Quan trọng là trong `M2_lm`, coefficient estimate (không được báo cáo) cho level cuối của `mnth` không bằng 0; thay vào đó, nó bằng số âm của tổng các coefficient estimate cho mọi level còn lại. Tương tự, trong `M2_lm`, coefficient estimate cho level cuối của `hr` là số âm của tổng các coefficient estimate cho mọi level còn lại. Điều này có nghĩa các coefficient của `hr` và `mnth` trong `M2_lm` luôn có tổng bằng 0, và có thể được diễn giải là độ chênh so với mean level. Ví dụ, coefficient -46.087 cho tháng Một cho biết rằng, khi giữ mọi variable khác cố định, thông thường có ít hơn 46 rider vào tháng Một so với mức trung bình hằng năm.

Cần nhận ra rằng lựa chọn coding thực sự không quan trọng, miễn là ta diễn giải model output đúng theo coding được sử dụng. Ví dụ, ta thấy prediction từ linear model là như nhau bất kể coding:

```
In [69]: np.sum(M_lm.fittedvalues - M2_lm.fittedvalues)**2
```

```
Out[69]: 1.53e-20
```

Tổng squared difference bằng 0. Ta cũng có thể thấy điều này bằng function `np.allclose()`:

```
np.allclose()
```

```
In [70]: np.allclose(M_lm.fittedvalues , M2_lm.fittedvalues)
```

```
Out[70]: True
```

Để tái tạo phía bên trái của Hình 4.13, trước hết ta phải lấy các coefficient estimate gắn với `mnth`. Các coefficient từ tháng Một đến tháng Mười Một có thể lấy trực tiếp từ object `M2_lm`. Coefficient của tháng Mười Hai phải được tính tường minh bằng số âm của tổng tất cả các tháng còn lại. Trước tiên ta extract mọi coefficient của month từ các coefficient của `M2_lm`.

```
In [71]: coef_month = S2[S2.index.str.contains('mnth')]['coef']
coef_month
```

```
Out[71]: mnth[Jan]      -46.0871
mnth[Feb]      -39.2419
mnth[March]    -29.5357
mnth[April]     -4.6622
mnth[May]      26.4700
mnth[June]     21.7317
mnth[July]     -0.7626
mnth[Aug]       7.1560
mnth[Sept]     20.5912
mnth[Oct]      29.7472
mnth[Nov]      14.2229
Name: coef, dtype: float64
```

Tiếp theo, ta append Dec dưới dạng số âm của tổng mọi tháng còn lại.

```
In [72]: months = Bike['mnth'].dtype.categories
coef_month = pd.concat([
    coef_month,
    pd.Series([-coef_month.sum()],
              index=['mnth[Dec]'],
              )
])
coef_month
```

```
Out[72]: mnth[Jan]      -46.0871
mnth[Feb]      -39.2419
mnth[March]    -29.5357
mnth[April]     -4.6622
mnth[May]      26.4700
mnth[June]     21.7317
mnth[July]     -0.7626
mnth[Aug]       7.1560
mnth[Sept]     20.5912
mnth[Oct]      29.7472
mnth[Nov]      14.2229
mnth[Dec]       0.3705
Name: coef, dtype: float64
```

Cuối cùng, để plot gọn hơn, ta chỉ dùng chữ cái đầu tiên của mỗi tháng, tức entry thứ 6 của mỗi label trong index.

```
In [73]: fig_month, ax_month = subplots(figsize=(8,8))
x_month = np.arange(coef_month.shape[0])
ax_month.plot(x_month, coef_month, marker='o', ms=10)
ax_month.set_xticks(x_month)
ax_month.set_xticklabels([l[5] for l in coef_month.index], fontsize
```

```

=20)
ax_month.set_xlabel('Month', fontsize=20)
ax_month.set_ylabel('Coefficient', fontsize=20);

```

Việc tái tạo plot phía bên phải trong Hình 4.13 theo một process tương tự.

```

In [74]:
coef_hr = S2[S2.index.str.contains('hr')]['coef']
coef_hr = coef_hr.reindex(['hr[{}]'.format(h) for h in range(23)])
coef_hr = pd.concat([coef_hr,
                    pd.Series([-coef_hr.sum()], index=['hr[23]'])
                    ])

```

Bây giờ ta tạo hour plot.

```

In [75]:
fig_hr, ax_hr = subplots(figsize=(8,8))
x_hr = np.arange(coef_hr.shape[0])
ax_hr.plot(x_hr, coef_hr, marker='o', ms=10)
ax_hr.set_xticks(x_hr[::2])
ax_hr.set_xticklabels(range(24)[::2], fontsize=20)
ax_hr.set_xlabel('Hour', fontsize=20)
ax_hr.set_ylabel('Coefficient', fontsize=20);

```

Poisson Regression

Bây giờ thay vào đó ta fit một Poisson regression model cho dữ liệu Bikeshare. Hầu như không có gì thay đổi, ngoại trừ việc giờ đây ta dùng function `sm.GLM()` với Poisson family được chỉ định:

```

In [76]:
M_pois = sm.GLM(Y, X2, family=sm.families.Poisson()).fit()

```

Ta có thể plot các coefficient gắn với `mnth` và `hr` để tái tạo Hình 4.15. Trước hết ta hoàn thiện các coefficient này như trước.

```

In [77]:
S_pois = summarize(M_pois)
coef_month = S_pois[S_pois.index.str.contains('mnth')]['coef']
coef_month = pd.concat([coef_month,
                    pd.Series([-coef_month.sum()],
                                index=['mnth[Dec]'])])
coef_hr = S_pois[S_pois.index.str.contains('hr')]['coef']
coef_hr = pd.concat([coef_hr,
                    pd.Series([-coef_hr.sum()],
                                index=['hr[23]'])])

```

Việc plot giống như trước.

```

In [78]:
fig_pois, (ax_month, ax_hr) = subplots(1, 2, figsize=(16,8))
ax_month.plot(x_month, coef_month, marker='o', ms=10)
ax_month.set_xticks(x_month)
ax_month.set_xticklabels([l[5] for l in coef_month.index], fontsize=20)
ax_month.set_xlabel('Month', fontsize=20)
ax_month.set_ylabel('Coefficient', fontsize=20)
ax_hr.plot(x_hr, coef_hr, marker='o', ms=10)
ax_hr.set_xticklabels(range(24)[::2], fontsize=20)
ax_hr.set_xlabel('Hour', fontsize=20)
ax_hr.set_ylabel('Coefficient', fontsize=20);

```

Ta so sánh fitted value của hai model. Fitted value được lưu trong attribute `fittedvalues` do method `fit()` trả về cho cả linear regression

fit lần Poisson fit. Linear predictor được lưu dưới attribute `lin_pred`.

In [79]:

```
fig, ax = subplots(figsize=(8, 8))
ax.scatter(M2_lm.fittedvalues ,
           M_pois.fittedvalues,
           s=20)
ax.set_xlabel('Linear Regression Fit', fontsize=20)
ax.set_ylabel('Poisson Regression Fit', fontsize=20)
ax.axline([0,0], c='black', linewidth=3,
          linestyle='--', slope=1);
```

Các prediction từ Poisson regression model có tương quan với các prediction từ linear model; tuy nhiên, prediction của Poisson model là non-negative. Vì vậy, Poisson regression prediction có xu hướng lớn hơn linear model prediction ở cả những mức ridership rất thấp lẫn rất cao.

Trong subsection này, ta fit Poisson regression model bằng function `sm.GLM()` với argument `family=sm.families.Poisson()`. Trước đó trong lab, ta dùng function `sm.GLM()` với `family=sm.families.Binomial()` để thực hiện logistic regression. Các lựa chọn khác cho argument `family` có thể được dùng để fit những loại GLM khác. Chẳng hạn, `family=sm.families.Gamma()` fit một Gamma regression model.